

**.EPIC & .EPICX  
SPECIFICATION**

**v1.2**

# WHAT IS EPIC?

## **EPIC = Extended Performance & Intelligent Cues**

EPIC is a lightweight, human-readable plain-text format for lyrics, scripting, AI generation, live-performance instructions, and timed cue alignment.

# .epic spec:

A structured plain-text authoring format for lyrics, scripting, generation metadata, and production intent.

Includes:

- Header
  - optional features, such as: BPM, Key, etc
  - optional subsection for AI/Generation definitions
- Body
  - line breaks indicate phrasing

Extension is “.epic”.

For compatibility with older systems, the “.epic.txt” variant is also accepted.

Example: “sample.epic.txt”

1

1a

---  
Title: Elem-en Ello Vo  
Artist: Kozzality

1b

[Generation]  
Styles: Edm, Goth, Industrial, Dubstep, Techno, Phonk  
Persona: Just Kozz  
---

2

[Verse]  
Epulote row span nisa con  
Ando go modo tray yo spawn  
Vosawn gane van cola mane  
Voss train man ola sane

[Chorus]  
Melon car stay voy  
Duran day  
Daylo cross ello joy  
Dagra ba yay  
Elon shayow  
Colagris en ello vo

[Verse 2]  
Enjelo base rico swan  
Carmelo daise micro vawn  
Torestry hawn ariso  
Comelise ray marino

[Chorus]  
Melon car stay voy  
Duran day  
Daylo cross ello joy  
Dagra ba yay  
Elon shayow  
Colagris en ello vo

[Bridge]  
Guess row heart  
Meya cart ay  
Terro part  
Genna karte  
...karte  
Enrin arte!

[Chorus]  
Melon car stay voy (melon karte)  
Duran day  
Daylo cross ello joy (ello joy)  
(Dagra ba yay)  
(Hey, hey)  
Colagris en ello vo  
Colagris...  
en ello vo!



Header section (metadata). Encapsulated by three ASCII hyphen-minus characters (-) on their own line (above and below.)

Required fields: Title, Authorship [Creator | Artist | Author]

Possible Fields:

- **1a** (main section)

- Title
- Authorship [Creator | Artist | Author]
- BPM
- Key
- TimeSignature
- Language: ISO Code
- Tags
- Offset (offset to be applied to timing)
- Production (production notes)
- Version:  
document version. Acceptable values:
  - numeric semantic version (e.g., 1.0, 1.2)
  - descriptive label (Draft, Final, Radio Edit, etc)
- EPICVersion: lyric spec version (1.0, 2.0, etc.)

- **1b** (AI/Generation subsection)

- Styles: styles prompt and generation instructions
- Persona: name of persona to be used
- Cover: title of song to be covered
- VocalGender: Male, Female, Neutral, Mixed
- Weirdness: 0 - 1.00
- StyleInfluence: 0 - 1.00
- Energy: 0 - 1.00
- TempoHint: descriptive

## 2 -

Body — the line entries.

Typical sections:

- Verse: Verse, Verse 2, etc.
- Pre-Chorus
- Chorus
- Bridge
- Intro
- Outro
- Final Chorus
- Spoken
- Custom (any custom sections are allowed)

**.epicx spec:** A timed plain-text performance format for synchronized lyrics, cue alignment, contextual instructions, and optional sub-event timing. Entries may contain readable payload lines, timed micro-events, or both.

Includes:

- Header
  - optional features, such as: BPM, Key, etc
  - optional subsection for AI/Generation definitions
- Body
  - numeric identifier
  - timestamp
    - optional: time range using MM:SS.mmm --> MM:SS.mmm (minutes, seconds, milliseconds)
  - Section (Verse, Chorus, etc)
  - line entry, micro-events: line breaks within a line entry indicate screen layout only and do not imply phrasing.

Extension is “.epicx”.

For compatibility with older systems, the “.epicx.txt” variant is also accepted.

Example: “sample.epicx.txt”

1

1a

---  
Title: Elem-en Ello Vo  
Artist: Kozzality

1b

[Generation]  
Styles: Edm, Goth, Industrial, Dubstep, Techno, Phonk  
Persona: Just Kozz  
---

2

1  
00:48.633  
[Verse]  
Epulote row span nisa con

2  
00:51.667  
[Verse]  
Ando go modo tray yo spawn

3  
00:55.033  
[Verse]  
Vosawn gane van cola mane

4  
00:58.400  
[Verse]  
Voss train man ola sane

5  
01:09.333  
[Chorus]  
Melon car stay voy

6  
01:12.067  
[Chorus]  
Duran day

7  
01:13.167  
[Chorus]  
Daylo cross ello joy

8  
01:15.567  
[Chorus]  
Dagra ba yay

9  
01:17.333  
[Chorus]  
Elon shayow



# 1 -

Header section (metadata). Encapsulated by three ASCII hyphen-minus characters (-) on their own line (above and below.)

Required fields: Title, Authorship [Creator | Artist | Author]

Possible Fields:

- **1a** (main section)
  - Title
  - Authorship [Creator | Artist | Author]
  - BPM
  - Key
  - TimeSignature
  - Language: ISO Code
  - Tags
  - Offset (offset to be applied to timing)
  - Production (production notes)
  - Version:
    - document version. Acceptable values:
      - numeric semantic version (e.g., 1.0, 1.2)
      - descriptive label (Draft, Final, Radio Edit, etc.)
    - EPICVersion: epic spec version (1.0, 2.0, etc.)
- **1b** (AI/Generation subsection)
  - Styles: styles prompt and generation instructions
  - Persona: name of persona to be used
  - Cover: title of song to be covered
  - VocalGender: Male, Female, Neutral, Mixed
  - Weirdness: 0 - 1.00
  - StyleInfluence: 0 - 1.00
  - Energy: 0 - 1.00
  - TempoHint: descriptive

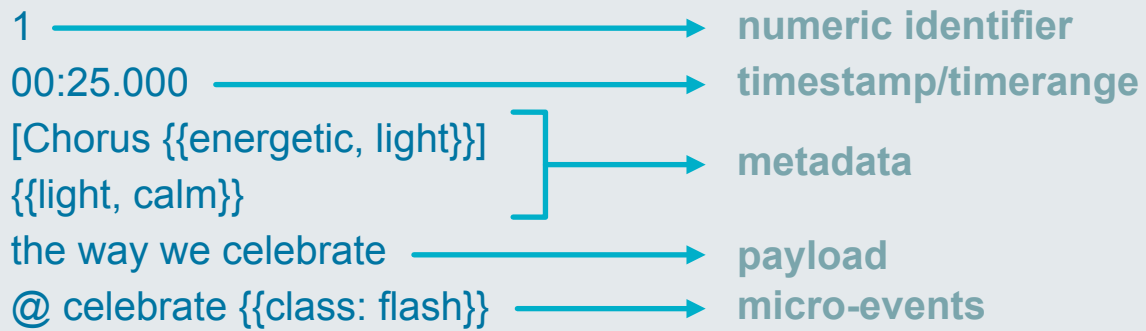
# 2 -

Body: Timed entries

- Numeric Identifier
- Timestamp
- Metadata - Section (optional)
- Metadata - Contextual Instructions (when applicable)
- Payload - line entry, standalone instruction block
- micro-events

# .EPICX ENTRIES

## STRUCTURE OF AN .EPICX ENTRY



# INSTRUCTION BLOCKS-1

Instruction blocks provide additional interpretive or structural information associated with line entries, sections, or performance timing.

Instruction blocks are enclosed in **double curly braces**:

```
{{ instruction }}
```

Instruction blocks are designed to be human-readable while remaining machine-parseable. They may describe pronunciation, vocal delivery, stylistic intent, production cues, or timing information.

Instruction blocks **do not replace line content**, but instead annotate or guide interpretation of the text.

## INSTRUCTION BLOCK TYPES

Instruction blocks may appear in three contexts.

### 1. Sectional Instruction

A sectional instruction applies to an entire section and is attached to a section label.

Example:

```
[Intro {{choral chant}}]
```

This instruction applies to the section that follows.

# INSTRUCTION BLOCKS-2

## 2. Inline Instruction

An inline instruction applies to nearby line entry content and appears within a line entry.

Example:

The Emerald Sea {{male singer}}

Inline instruction may describe vocal delivery, pronunciation, or expressive intent.

## 3. Standalone Instruction

A standalone instruction appears on its own line and applies to the surrounding performance context.

Example:

{{guitar solo}}

Standalone instructions are commonly used for musical performance cues.

# INSTRUCTION BLOCKS-3

## INSTRUCTION CONTENT

Instruction blocks contain one or more **instruction item** separated by commas.

Example:

```
{{whispered, distant}}
```

Instruction items may take the form of:

### Phrase Instruction

A descriptive phrase.

Example:

```
{{whispered}}
```

Instruction items may take the form of:

### Key/Value Instruction

A structured instruction consisting of a key and value.

Example:

```
{{pronounced: /ter/}}
```

# INSTRUCTION BLOCKS-4

Multiple key/value instructions may appear within a single block:

```
{{pronounced: /ter/, emotion: sorrowful}}
```

## Instruction Parsing

instruction blocks follow the general structure:

```
{{ instruction_item, instruction_item2, ... }}
```

Whitespace following commas is optional but recommended for readability.

Instruction blocks may contain a mixture of phrase instructions and key/value instructions.

# INSTRUCTION BLOCKS-5

## Treatment Of Whitespace

Inside an instruction block, item boundaries are defined only by commas. Whitespace, including line breaks, is ignored except as separation between words in a text value. Keys must not contain whitespace. Values and free-text items may contain whitespace. An unquoted colon is reserved exclusively as the key/value separator and may appear only once in an item. Empty items are not allowed. Empty values are not allowed.

```
{  
    social: like and subscribe,  
    instruction item,  
    keyItem2: some value  
}
```

# INSTRUCTION BLOCKS-6

## String Literals

Quoted string literals enclosed in backticks preserve literal characters, such as commas and colons.

```
{  
  social: `like, comment: subscribe`,  
  instruction item,  
  keyItem2: `longer value including line breaks,  
  tabs, commas and so on`  
}
```

In this example, the commas, colons, and whitespace characters within the backticks are treated as literal text.

## FORWARD COMPATIBILITY

Instruction blocks are designed to be extensible.

Parsers SHOULD ignore unknown instruction keys or phrases that are not recognized. This allows future extensions without breaking compatibility with existing implementations.



# MICRO-EVENTS-1

Micro-events apply to .epicx docs only, but can be created as references in .epic docs.

Micro-events are timed sub-events introduced by “@”, optionally followed immediately by a timestamp. If no timestamp is provided, the timestamp defaults to the containing entry anchor’s start time.

Example:

```
@00:51.000 {{lights dim}}
```

Example using instruction blocks, performance cues, micro-events:

```
3
00:49.566 --> 00:55.777
[Bridge]
{{vocals: choir}}
the way
we celebrate
@00:51.000 {{lights dim}} {{duration: 1.2 sec}}
@00:52.000 {{fire pops}}
@00:52.000 {{cello swells}}
```

Example using micro-events for word-timing:

```
4
00:49.566 --> 00:51.777
The way
we celebrate
@00:51.000 The
@00:51.120 way {{br}}
@00:51.300 we
@00:51.420 celebrate
```

# MICRO-EVENTS-2

Rules for writing micro-events:

## 1. Micro-event containment

If an entry anchor is a time range, all micro-event timestamps must fall within that range.

## 2. Micro-event scope

Micro-events are scoped to the containing entry and are not valid as top-level standalone records.

## 3. Word-timing

Word timing requires standard payload. Then the micro-events assign word timing signatures (timestamps)

## 4. Word-timing line breaks

`{{br}}` is reserved and inserts a rendered line break immediately after the associated word/content.

# MICRO-EVENTS-3

## 1. Micro-event chronology

Within a containing entry, explicit micro-event timestamps must appear in chronological order. Micro-events with omitted local timestamps inherit the containing entry anchor's start-time relevance.

Only explicit micro-event timestamps are subject to chronology validation within the containing entry.

```
3
00:49.566 --> 00:55.777
[Bridge]
{{vocals: choir}}
the way we celebrate
@00:51.000 {{lights dim}} {{duration: 1.2 sec}}
@00:52.000 {{fire pops}}
@00:52.000 {{cello swells}}
```

With implicit micro-events:

```
3
00:49.566 --> 00:55.777
[Bridge]
{{vocals: choir}}
the way we celebrate
@ {{lights dim}} {{duration: 1.2 sec}}
@ {{fire pops}}
@ {{cello swells}}
```

# MICRO-EVENTS-4

## Micro-event references in .epic docs

Micro-events can be authored in .epic docs using “@”references inside instruction blocks.

Example:

```
Waves die down inside my chest {{@lights dim}}
```

...which converts to .epicx as:

```
1
00:35.000
Waves die down inside my chest
@ {{lights dim}}
```

Micro-events can be expressed alongside other instruction block content:

```
Waves die down inside my chest {{volume decreases, @lights dim}}
```

.epicx:

```
1
00:35.000
{{volume decreases}}
Waves die down inside my chest
@ {{lights dim}}
```

# MICRO-EVENTS-5

## Two Flavors

Micro-events can be seen as being two types:

- simple events
- payload modifiers

## As A Simple Event

Event triggers:

```
1
00:35.000
Waves die down inside my chest
@ {{lights dim}}
```

## As Payload Modifiers

The target becomes the payload, itself. Word timing is an example of this:

```
1
00:35.000
Waves die down inside my chest
@ 00:35.000 Waves
@ 00:35.178 die
...
```

# MICRO-EVENTS-6

## As Payload Modifiers (cont'd)

Some vendors can take something like:

```
Waves die down inside my chest {{@waves: warp}}
```

...and convert it to:

```
1  
00:35.000  
Waves die down inside my chest  
@ Waves {{class: warp}}  
...
```

# EBNF-1

## COMMON LEXICAL DEFINITIONS

```
newline      = "\r\n" | "\n" ;
separator    = newline , {newline} ;
strict_separator = newline ;
ws           = " " ;

digit        = "0" | "1" | "2" | "3" | "4"
              | "5" | "6" | "7" | "8" | "9" ;

alpha        = ? any unicode letter ? ;
alphanum     = alpha | digit ;

text_char    = ? any character except newline ? ;
text_line    = { text_char } ;
punctuation = "." | "," | "!" | "?" | ":" | ";" ;

hyphen_line  = "----" , newline ;

float_0_1    = "0"
              | "1"
              | "0." , digit , digit
              | "1.00" ;

integer      = digit , { digit } ;

freeform_body = free_lines_with_text ;
free_lines_with_text = free_line_run , nonempty_line , free_line_run ;

free_line_run = { empty_line | nonempty_line } ;
empty_line    = newline ;
nonempty_line = nonempty_text_line , newline ;
nonempty_text_line = text_char , { text_char } ;

eof          = ? end of file ? ;
```

# EBNF-2

## TIMESTAMP (EPICX)

```
timestamp      = short_timestamp | long_timestamp ;
time_range     = timestamp , ws , "-->" , ws , timestamp ;

short_timestamp = minute_2 , ":" , second , "." , millisecond ;
long_timestamp  = hour_2 , ":" , minute_2 , ":" ,
                  second , "." , millisecond ;

hour_2         = digit , digit ;
minute_2       = digit , digit ;
second         = digit , digit ;
millisecond     = digit , digit , digit ;
```



# EBNF-3

## HEADER

```
header = hyphen_line ,
        title_field ,
        authorship_group ,
        { header_optional_field } ,
        [ strict_separator , generation_block ] ,
        hyphen_line ;
```

```
authorship_group      =
                        creator_only
                        | artist_only
                        | author_only
                        | creator_artist
                        | creator_author
                        | artist_creator
                        | artist_author
                        | author_creator
                        | author_artist
                        | creator_artist_author
                        | creator_author_artist
                        | artist_creator_author
                        | artist_author_creator
                        | author_creator_artist
                        | author_artist_creator ;
```

```
creator_only          = creator_field ;
artist_only            = artist_field ;
author_only            = author_field ;
```

```
creator_artist        = creator_field , artist_field ;
creator_author         = creator_field , author_field ;
artist_creator         = artist_field , creator_field ;
artist_author          = artist_field , author_field ;
author_creator         = author_field , creator_field ;
author_artist          = author_field , artist_field ;
```

```
creator_artist_author  = creator_field , artist_field , author_field ;
creator_author_artist  = creator_field , author_field , artist_field ;
artist_creator_author  = artist_field , creator_field , author_field ;
artist_author_creator  = artist_field , author_field , creator_field ;
author_creator_artist  = author_field , creator_field , artist_field ;
author_artist_creator  = author_field , artist_field , creator_field ;
```

# EBNF-4

## HEADER (CONT'D)

```
header_optional_field = bpm_field
                        | key_field
                        | timesignature_field
                        | language_field
                        | tags_field
                        | offset_field
                        | production_field
                        | version_field
                        | epicversion_field ;

title_field            = "Title:" , ws , text_line , newline ;

creator_field          = "Creator:" , ws , text_line , newline ;
artist_field           = "Artist:" , ws , text_line , newline ;
author_field           = "Author:" , ws , text_line , newline ;

bpm_field              = "BPM:" , ws , text_line , newline ;
key_field              = "Key:" , ws , text_line , newline ;
timesignature_field    = "TimeSignature:" , ws , text_line , newline ;
language_field         = "Language:" , ws , text_line , newline ;
tags_field             = "Tags:" , ws , tag , { ",", ws , tag } , newline ;
tag                    = tag_character , { tag_character } ;
tag_character           = alpha | digit | "-" | " " ;

offset_field           = "Offset:" , ws , offset_value , newline ;
offset_value           = two_digits , ":" , two_digits , "." , three_digits ;
two_digits              = digit , digit ;
three_digits            = digit , digit , digit ;

version_field          = "Version:" , ws , text_line , newline ;
epicversion_field      = "EPICVersion:" , ws , text_line , newline ;

production_field       = "Production:" ,
                        ( ws , text_line , newline
                          | ws , newline
                          | newline ) ,
                        freeform_body ;
```

# EBNF-5

## GENERATION BLOCK

```
generation_block =  
    "[Generation]" , newline ,  
    { generation_field } ;
```

```
generation_field =  
    styles_field  
    | usestyle_field  
    | persona_field  
    | cover_field  
    | vocalgender_field  
    | weirdness_field  
    | styleinfluence_field  
    | energy_field  
    | tempohint_field
```

```
styles_field          = styles_single | styles_multiple ;
```

```
styles_single         = "Styles:" , freeform_body ;
```

```
styles_multiple       = "Styles:" ,  
                        { ws } , newline ,  
                        style_option ,  
                        { strict_separator , style_option } ;
```

```
styles_text           = { text_line_or_empty } ;
```

```
text_line_or_empty   = text_line | newline ;
```

```
style_option_mk       = integer , "." , ws ;
```

```
style_option          = style_option_mk ,  
                        text_line , newline ;
```

```
usestyle_field        = "UseStyle" , ":" , ws , integer , newline ;
```

```
persona_field         = "Persona:" , ws , text_line , newline ;
```

```
cover_field           = "Cover:" , ws , text_line , newline ;
```

```
vocalgender_field     = "VocalGender:" , ws ,  
                        ( "Male" | "Female" | "Neutral" | "Mixed" ) ,  
                        newline ;
```

```
weirdness_field       = "Weirdness:" , ws , float_0_1 , newline ;
```

```
styleinfluence_field  = "StyleInfluence:" , ws , float_0_1 , newline ;
```

```
energy_field          = "Energy:" , ws , float_0_1 , newline ;
```

```
tempohint_field       = "TempoHint:" , ws , text_line , newline ;
```

# EBNF-6

## SECTION LABELS

```
known_section_name    = "Verse"  
                        | "Chorus"  
                        | "Pre-Chorus"  
                        | "Bridge"  
                        | "Intro"  
                        | "Outro"  
                        | "Final Chorus"  
                        | "Spoken" ;  
  
custom_section_name   = custom_section_char ,  
                        { custom_section_char } ;  
  
custom_section_char   = ? any character except "]" and newline ? ;  
  
section_index         = ws , integer ;  
  
known_section_label   = known_section_name ,  
                        [ section_index ] ;  
  
custom_section_label  = custom_section_name ;  
  
section_label         = "[" ,  
                        ( known_section_label | custom_section_label ) ,  
                        [ ws , instruction_block ] ,  
                        "]" ;
```

# EBNF-7

## INSTRUCTION BLOCKS

```
instruction_block      = "{{" ,
                        ib_ws , instruction_item_list , ib_ws ,
                        "}}" ;

instruction_item_list  = instruction_item ,
                        { ib_ws , "," , ib_ws , instruction_item } ;

instruction_item       = key_value_item | text_item ;

key_value_item         = instruction_key ,
                        ib_ws , ":", ib_ws , text_value ;

text_item              = text_value ;

instruction_key         = key_char , { key_char } ;

text_value             = text_part , { ib_ws , text_part } ;

text_part              = string_literal | text_word ;

string_literal         = "\"" , string_char , { string_char } , "\"" ;

string_char            = ? any character except "\"" ? ;

key_char               = ? any character except space,
                        tab, newline, ":", ",", "{", "}", "\"" ? ;

text_word              = ib_text_char , { ib_text_char } ;

ib_text_char           = ? any character except space,
                        tab, newline, ":", ",", "{", "}", "\"" ? ;

ib_ws                  = { " " | "\t" | newline } ;
```

# EBNF-8

## .EPIC GRAMMAR

```
epic_document    = header ,  
                  separator ,  
                  [ epic_body ] ,  
                  eof ;  
  
epic_body         = { epic_section } ;  
  
epic_section      = section_label , newline ,  
                  { epic_line } ;  
  
epic_line         = epic_text_line , [ ws , instruction_block ] , newline ;  
  
epic_text_line    = { epic_text_char } ;  
  
epic_text_char    = ? any character except newline, or a "{" sequence ? ;
```

# EBNF-9

## .EPICX GRAMMAR

```
epicx_document      = header ,  
                      [ strict_separator , epicx_body ] ,  
                      eof ;  
  
epicx_body          = timed_entry ,  
                      { strict_separator , timed_entry } ;  
  
entry_index         = integer ;  
  
timed_entry         = entry_index , newline ,  
                      entry_anchor , newline ,  
                      [ section_label , newline ] ,  
                      entry_line ,  
                      { entry_line } ;  
  
entry_anchor        = timestamp | time_range ;  
  
entry_line          = micro_event_line | payload_line ;  
  
micro_event_line    = "@" ,  
                      ( timestamp , ws , line_content  
                        | ws , line_content ) ,  
                      newline ;  
  
payload_line        = line_content ,  
                      newline ;  
  
line_content        = line_item ,  
                      { ws , line_item } ;  
  
line_item           = instruction_block | text_fragment ;  
  
text_fragment       = text_fragment_char ,  
                      { text_fragment_char } ;  
  
text_fragment_char  = ? any character except newline, "{", "}" ? ;
```

# RULE SUMMARY-1

## HEADER

1. Defined by three ASCII hyphen-minus characters (-) on their own line (above and below)
2. Title and Authorship [Creator | Artist | Author] are required. In the case of multiple, for systems, the first listed takes precedence
3. Title and Authorship fields must appear first, in that order
4. Optional fields follow. Each label should be capitalized, by convention
  - a. BPM (recommended all-caps)
  - b. Key
  - c. TimeSignature (Pascalcase)
  - d. Language: ISO Code
  - e. Tags
  - f. Offset (offset to be applied to timing)
  - g. Production (production notes)
  - h. Version
  - i. EPICVersion ( capitalized exactly as shown)
  - j. By convention, “Version” and “EPICVersion” should appear last in the optional section, and in that order
5. Each header property MAY appear at most once unless explicitly defined as repeatable
6. A Production notes block continues until the next header property, the [Generation] section, or the closing header delimiter (---)
7. The “Generation” section must come last within the header block
8. “Generation” must be capitalized exactly as shown
9. “Generation” is optional and may be empty
10. Field order “Generation” is unrestricted
11. “Styles” within “Generation” is freeform and may include one or multiple empty lines
12. A single blank line must precede “Generation”



# RULE SUMMARY-2

## BODY

### .EPIC

1. Sections are identified by labels using square brackets. The label should be capitalized. Example: [Verse]
2. Subsequent verses are numbered by order (e.g., [Verse], [Verse 2], [Verse 3])
3. Typical labels include:
  - a. Intro
  - b. Verse
  - c. Verse 2, Verse 3, etc
  - d. Pre-Chorus
  - e. Chorus
  - f. Bridge
  - g. Final Chorus
  - h. Outro
4. Section can be empty
5. By convention, it is recommended that there be an empty line between sections
6. Performance instructions can be added using “{{“ + instructions +”}}”. Example:

You will find

The Emerald Sea {{male singer}}

Performance instruction can be either:

- a. Sectional Instruction : [Intro {{choral chant}}]
  - b. Inline Instruction: The Emerald Sea {{male singer}}
  - c. Standalone (performance) Instruction: {{big drop}}
7. File extension is “.epic”. “.epic.txt” can be used for compatibility with older systems.

# RULE SUMMARY-3

## BODY

### .EPICX

1. Timed entries must follow this order:
  - a. Entry number.  
Entry numbers must:
    - i. Start at 1
    - ii. Increase sequentially by 1
    - iii. Be unique
    - iv. Appear in ascending order
  - b. Timestamp
  - c. Section label (optional)
  - d. Entry line(payload), micro-events
2. Entry lines may span multiple lines.
3. Entry lines must not contain empty lines, except at the end of an entry
4. Each entry must be separated by exactly one empty line
5. Timestamp format must be either MM:SS.mmm or HH:MM:SS.mmm  
Use HH:MM:SS.mmm when hour-based mapping is needed. 60 min. ceiling is enforced, making 99 min. become 01:39:00.000 rather than 00:99:00.000
6. An optional exit timestamp may be defined using the pattern:  
entry timestamp --> exit timestamp (two dashes followed by a greater-than symbol between timestamps)
7. File extension is ".epicx". ".epicx.txt" can be used for compatibility with older systems.

# RULE SUMMARY-4

## BODY

### .EPICX (CONT'D)

8. Micro-events may be included (defined as “@” + timestamp, or just “@”):

1

00:48.633

[Verse]

Epulote row span nisa con

@00:48.633 Epulote

@00:48.900 row

@00:49.200 span {{br}}

@00:49.450 nisa

@00:49.750 con

9. {{br}} is reserved and inserts a rendered line break immediately after the associated content (as seen above). Only used for rendering timed-word formatting.

# QUICK LOOK

## HEADERS

- Title
- Authorship [Creator | Artist | Author]
- BPM
- Key
- TimeSignature
- Language
- Tags
- Offset
- Production
- Version
- EPICVersion

## GENERATION

- Styles
- UseStyle
- Persona
- Cover
- VocalGender
- Weirdness
- StyleInfluence
- TempoHint

# LICENSING

Copyright 2026 Jade Rigby

Licensed under the Apache License, Version 2.0 (the "License");  
you may not use this file except in compliance with the License.  
You may obtain a copy of the License at:

<http://www.apache.org/licenses/LICENSE-2.0>